RATIONALE

Why one shared document, and what it costs to skip it

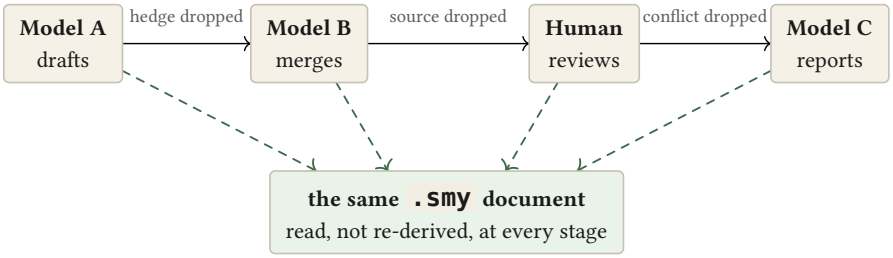
Vladimir Ulogov · 2026 · smysl 0.8.0 · format smysl/0.1 · kernel smysl.kernel/0.1

This is a document about a narrow, specific problem: what happens to a claim as it is handed from one system to the next — model to model, model to person, person back to model — and why the usual fix for that problem, a bespoke wire format between each pair of systems, only ever solves half of it. `smysl` is a plain-text document format built to close the other half. This document makes the case for it twice: once in the terms anyone doing this work already reasons in, and once in the terms information theory gives a name to. The two are the same argument; the second is just precise about what the first was gesturing at.

The problem, plainly

When one AI system hands work to another, what actually crosses the wire is prose. The system that wrote it **had** a structure in mind while it worked — this rests on that, this is a guess, these two things disagree — and then collapsed all of it into paragraphs anyway, because paragraphs are the only thing on offer. The next system in the chain has to guess that structure back out of the prose, act on its guess, and then flatten its own version into prose again for whoever is next. Do this more than once and the same three things go missing every time:

- **Hedges vanish.** "The data suggests" becomes "the data shows" becomes "we found" — a chain of retellings, each one slightly more confident than the one before it, with nobody having earned the extra confidence.
- **Sources evaporate.** By the third hop nobody can say where a number came from, because nothing about a citation survives being paraphrased.
- **Disagreement disappears.** Two conflicting findings go in; whichever one the last system preferred comes out. The conflict is not resolved — it is just dropped, and the output looks unanimous.



Top row: prose re-told four times, losing something specific at each hop. Bottom: one document, present unaltered at all four stages — there is no re-telling to lose anything in.

The obvious fix is a schema: agree on a JSON shape between each pair of systems and stop passing free text. That solves the problem for the machines and creates a new one for everyone else — nobody reviews a wire format, so the one place a person could have caught this drift is now the one place they cannot see. `smysl` takes a different position: make the artifact itself precise enough to carry hedges, sources and disagreement structurally, **and** plain enough that the same file is what a person opens to check the work.

What that costs, measured

The three losses above are the ones everybody who has run a multi-hop pipeline already believes in. It is worth knowing how large they actually are, because the answer is not "a little, at the margins" — and because a pipeline losing them gives no sign that it has. Prose does not announce what it dropped.

So: five documents, five hops each, summarised by a real model at every hop, run twice over — once through `gemini-3.5-flash-lite` and once through `deepseek-chat`. Ninety claims in total. A second model reads the output afterwards and is asked, for each original claim, whether the final text still states it, how confidently the **text** now puts it, and what the text says it rests on.

● ● ● *the evaluation harness, `make eval-live`*

	tokens	claims kept	hedges lost	sources kept
control (no summarising)	1.00	90/90	0/90	50/50
prose baseline	0.29	72/90	42/90	1/50
smysl	0.49	90/90	0/90	50/50

Read the bottom two rows against each other. The prose chain compresses **harder** than it was asked to — 0.29 of the original against smysl's 0.49 — and pays for it twice. Forty-two of ninety surviving claims came out the far end reading as measurements when the document that went in had called them inferred, cited or derived. And of the fifty claims that named where they came from, exactly one still named it.

The top row is what makes the other two mean anything. The same judge, the same prompt, reading the document **before** any summarising, recovers every hedge and every source — it declined to rule on none of the ninety. Whatever went missing over five hops went missing in the chain, not in the instrument that measured it.

> **THE POINT**
>
> Nothing here is a criticism of the models. Both were asked to summarise and both summarised well — the output reads fluently and says broadly the right things. Hedges and citations are simply not what prose is built to preserve, and a summariser has no way to know it dropped one. On the `smysl` side both numbers are structural rather than lucky: confidence is a field that a rule checks, and a source is a field that travels with whatever travels.

Two models, five documents, one run each — enough to say the effect is not one model's quirk, not enough to put an error bar on it.

What the document actually looks like

There is no separate machine version. This is a complete, real excerpt — the review copy and the wire format are the same bytes:

```
@evidence e/pool-wait { status: measured, source: { kind: metric, ref:
"pool.wait_ms" } }
~ Pool acquisition wait rose from 2 ms to 310 ms over the same window.

@claim c/pool-saturation { status: inferred, grounds: [e/pool-wait] }
~ The eu-west connection pool is saturated.

@rel c/canary-clean --rebuts--> c/pool-saturation { weight: 0.6 }
```

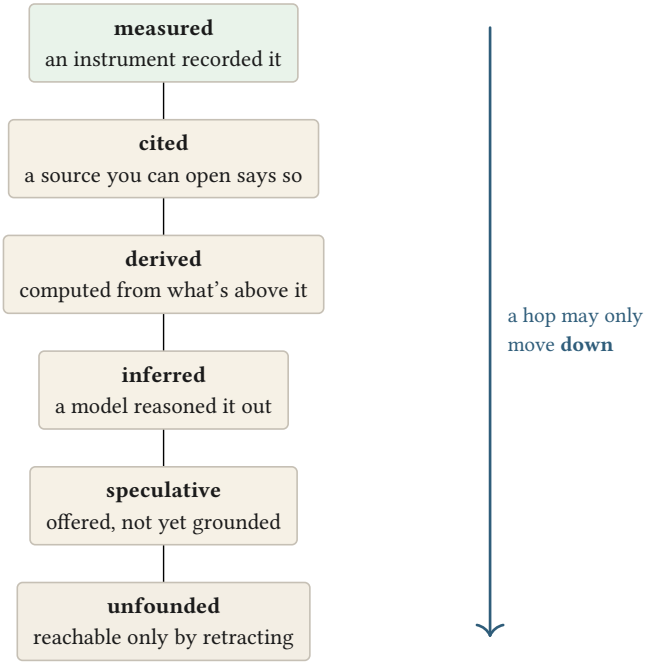
> **TERM** **Unit**
>
> A **unit** is one recorded thing — a piece of evidence, a claim, a finding, a definition. Every unit carries a **status** (how sure the document is entitled to be about it) and, where it rests on something else, a `grounds` list naming exactly which units it depends on. A `rel` records a relationship **between** units — `rebuts`, `causes`, `warrant` — including disagreement, which is a first-class thing to record rather than an anomaly to hide.

Three things are being carried here that a paragraph would have thrown away. First, **how sure** the document is entitled to be: `measured` (an instrument recorded it) is not the same claim as `inferred` (a model reasoned it out), and the difference is a field, not a tone of voice. Second, **what it rests on**: `grounds: [e/pool-wait]` means the claim stands or falls with that one piece of evidence — retract the evidence and the tool can tell you exactly what falls with it. Third, **what contradicts it**: the `rebuts` edge is part of the document, sitting right next to the claim it argues with, not filed away in a review thread nobody reads twice.

The trust ladder, and why it can only fall

Every unit's status sits on a six-rung ladder, and the document's central mechanical rule is about that ladder: a claim's status can be **weakened** as it crosses a hop, but never **strengthened**. A model reading

this document may propose at most `inferred`; if it writes `measured` anyway, the tool downgrades it and says so, before the claim ever reaches whoever reads it next.



Six rungs, checked mechanically on every unit: nothing here is a matter of tone or house style — it is validated the same way a type error is.

That single rule is doing more work than it looks like. It is the mechanism that stops the "the data suggests" → "we found" drift described above from ever compounding, because there is no step in the chain where confidence is allowed to go up, only ones where it is allowed to fall or hold.

The same rule, said precisely

Everything so far has been the practitioner's version of one argument: a hop must never come away knowing a claim more certainly than the hop before it did. Information theory has an exact name for that argument, and getting to it takes three short steps in order — what information **is**, what a multi-hop pipeline **is** as a mathematical object, and what necessarily happens to information as it crosses one.

> TERM **Information**
>
> Used here in Shannon's sense: information is not "content," it is the amount by which a message reduces someone's uncertainty about something. A coin flip you already saw carries no information; one you have not carries some. A claim that only restates what you already believed carries less information than one that would change what you are willing to act on.

A single message's information is not yet the interesting quantity, though — a pipeline is many messages end to end, each one built only from the one just before it. That shape has a name too.

> **TERM** **Markov chain**
>
> A chain of steps, `A → B → C`, in which each step depends only on the one immediately before it. Once you know `B`, learning more about `A` cannot tell you anything further about `C` that `B` did not already carry. A multi-hop pipeline behaves exactly this way: each hop only ever sees what the hop before it handed forward.

Model, then model, then person, then model again is a chain of exactly that shape. And once a chain has that shape, one inequality governs everything that is allowed to happen to information as it moves along it.

> **TERM** **Data Processing Inequality**
>
> For any chain `X → Y → Z`, the information `Z` carries about `X` can never exceed the information `Y` carried about `X`. Processing can preserve information or destroy it, but it cannot manufacture more of it than arrived. Photocopy a photocopy and you do not get a sharper original — you get a copy of a copy, at best.

> **THE POINT**
>
> `smysl`'s falling-only status rule **is** the Data Processing Inequality, applied to one number instead of a whole distribution, and checked on every document rather than trusted to whoever is doing the copying. This is not an analogy borrowed after the fact — it is what makes the rule correct engineering rather than a house style: no hop can legitimately know a claim more certainly than the hop before it did, so no hop is allowed to say so.

Compressing without losing the argument

The inequality above is a ceiling: it stops a hop from **inflating** what it is entitled to claim. It has nothing to say about a second, equally ordinary problem — every hop also has to fit inside a budget, which means throwing part of the document away **on purpose**. That is a different question, with its own exact answer, and it is the question `pack` exists to answer.

A document does not always travel whole. `pack` fits a document to a token budget, and it has to decide what survives the cut:

● ● ● *fixtures/corpus/F1-incident.smy, packed to 200 tokens*

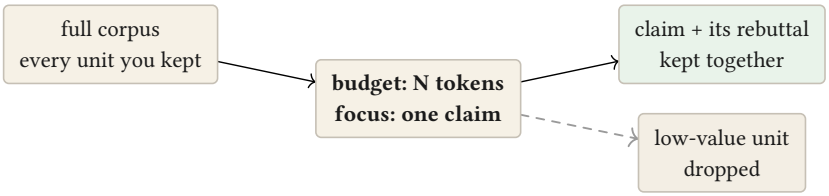
```
$ smysl --format surface pack --budget 200 --explain --focus c/pool-saturation
F1-incident.smy
b3:cvhirtgs2mpvli2ethhyeo32uf @L0  -  earned on density
b3:phsoomklkmlq3sjvbe6cyuqy5v @L0  C3  rebuts b3:cvhirtgs2mpvli2ethhyeo32uf
b3:xkys7j42mcuyiaxiyh73xddimr dropped: low-value
F1-incident.smy: 7 of 8 unit(s), 193 of 200 tokens, greedy mode, gap 0.011
```

Read the middle line again: that unit was kept **because** it rebuts one that was also kept. The two vocabulary items below are what make that not a coincidence of the implementation.

> TERM **Rate–distortion**
>
> Fix a budget of bits (the **rate**) and a way of scoring how far a compressed version departs from the original (the **distortion**). For any rate there is a best fidelity achievable — a real, computable number, not a rule of thumb. Spend fewer bits and the achievable fidelity only gets worse; there is a curve, and you choose where on it you stand, not whether it applies.

> TERM **Information bottleneck**
>
> A refinement of rate–distortion for when fidelity to the **whole** original does not matter — only fidelity to one downstream question does. The method compresses hard everywhere except along whatever predicts the answer to that one question, and lets the rest go without ceremony.

`--focus c/pool-saturation` names the question; `--budget 200` names the rate. What must survive is not "the most important-sounding sentences" but whatever the focus claim's answer actually depends on — which is why its rebuttal travels with it and a lower-density unit does not. A budget too small to hold both the claim and its objection does not quietly ship the claim alone and call it packed; the operation fails, loudly, on stderr.



*A budget too small to hold a claim **and** its rebuttal fails the pack outright, rather than shipping the claim looking uncontested.*

Merging without a referee

Packing decides what a **single** hop is allowed to keep. The last piece is what happens when **two** hops — two agents, working from the same corpus at the same time — hand back documents that were never reconciled with each other. Nothing above solves that; it needs its own operation.

Two agents can work on the same document at once, and neither has to wait for the other or ask a coordinator who goes first.

two independently-authored documents, combined

```
$ smysl merge agent-a.smy agent-b.smy -o combined.cbor
agent-b.smy: contention k/ccm3actwjjti65famnoe6mapo5d over
b3:cvhirtgs2mpvli2ethhyeo32uf
  (2 positions, live-rebuttal)
```

> TERM **Canonical identity**
>
> An identifier computed from **what a unit says**, not from where it sits in a file or who wrote it down. Two agents that independently record the exact same claim compute the exact same identifier without comparing notes, and a later merge collapses the two into one entry automatically — there is no registry to consult and nobody assigning ids.

> TERM **Join**
>
> From lattice math: given two partial pictures of the same situation, their **join** is the smallest single picture containing everything both of them knew — the least upper bound. A join is commutative and order-independent: it makes no difference which partial picture the merge sees first, and it never has to choose a winner to produce an answer.

Merging two `smysl` documents is a join over their units, keyed by canonical identity. Where the two genuinely disagree — not the same claim twice, but two different claims about the same thing — the join does not silently prefer whichever arrived second, the way an ordinary "last write wins" merge would. It records the disagreement as a **contention**, on the document, for a person to resolve deliberately. Nothing is picked for you behind your back.

Finding things again

A pipeline that has run for a while produces a store nobody has read. It holds units written by agents on other machines, named by hashes, and the question you arrive with is not "what does the graph say is important" but "where is the part about the connection pool".

`salience` cannot answer that. It ranks by structure — what grounds what, what the argument leans on — and never reads a word. So `find` ranks by words instead, over the **gist** of every unit.

That last detail is what makes it work at all. A unit's payload might be a stack trace, a table of measurements, a diff or a page of prose, and no single way of searching covers all four. But every unit carries a gist, because the format requires one, and a gist is a sentence about whatever the payload happens to be. You never search the telemetry. You search the sentence somebody — or some model — wrote about it.

It is worth saying plainly what this is not. It is lexical search: it matches words, not meanings. Measured over the reference corpus it is perfect on identifiers like `pool.wait_ms` and near-perfect when your words match the author's, and it is weak when they do not — a paraphrased query finds the right claim somewhere in the top five three times in four, but puts it first only once in eight. Evidence and data are found reliably because they name concrete things; claims are found less reliably because a claim is an interpretation, and interpretations get phrased differently by different people.

The ceiling has since been raised. A semantic backend sits behind the same seam, and the numbers say what it bought: on queries phrased differently from the text they are looking for, the right unit is now ranked first half the time rather than one time in eight. Where a query is a **name** — a metric, a path, a symbol — the lexical engine still answers it, perfectly, and the tool routes on the shape of the query rather than asking you to choose.

The first version of that routing was worse than either engine alone, and its test passed. That is recorded here rather than quietly fixed, because it is the same lesson the rest of this document keeps arriving at: a measurement you did not take is a decision you did not make.

The honest position is that this is a floor that has been raised once, and it was built as a seam so it can be raised again without disturbing anything beneath it. What it buys today is that a store stops being opaque, with no model, no index on disk, no network call, and the same answer on every machine.

What it buys when handed to `pack` is different in kind, and it is the part worth the attention. Retrieval on its own returns a ranked list, and the units at the top of one are usually claims. A claim without its grounds is an assertion — which is the ordinary failure of a retrieval-augmented pipeline, where a model receives five relevant-looking passages and is left to invent what connects them. It will invent something plausible, because that is what the input asks of it.

`pack --query` refuses to hand a claim over naked. The matched units become the focus, and packing pulls in what the format says is **required**: the grounds they rest on, the definitions they need to be read at all, and any live rebuttal — because a claim that travels without its rebuttal arrives looking uncontested. The result is not the next-most-similar passages. It is the argument, cut to a budget, with nothing load-bearing missing.

That composition is the thing neither half does alone, and it is a fair summary of the whole design: the format knows what supports what, so a tool built on it can answer a question without quietly dropping the reason the answer holds.

Where it sits in a pipeline

None of the above requires rebuilding a pipeline, and it is worth being concrete about what adopting it does and does not involve, because the honest answer is narrower than the argument might suggest.

There is exactly one place a model is needed: the entrance. `ingest` turns a document into units, and it is the only operation here that costs a model call. Everything downstream of it — fitting to a budget, merging two agents' work, ordering a reading, rendering an artifact, checking any of it — is ordinary computation over the graph. Same input, same output, byte for byte, no inference and no variance.

That is the shape of the trade. A pipeline that summarises at every hop pays a model at every hop and cannot tell you what each one discarded. A pipeline carrying `smysl` pays a model once at the boundary and then stops paying: the five hops in the measurement above cost five model calls on the prose side and zero on the other.

> TERM **The boundary**
>
> Model output does not enter the graph directly. It is parsed, checked against the rules above, and written to a staging file for a person or a later step to accept — `merge --staged` is that acceptance. A model overstating its confidence is corrected and the correction recorded, at the door, before anything downstream can rest on it.

Two further things follow from that boundary being the only model-dependent part. Instrument data does not go through it at all: `import` transcribes a table of readings directly, which is the only path allowed to record `measured`, because a tool copying a number is doing something a model reading a

document cannot. And adoption is incremental — one pipeline, or even one hop of one pipeline, produces documents that the rest of the system can keep treating as text, because that is what they are.

WHAT IT DOES NOT DO

The boundary is the one place with no guarantee behind it. A model converting prose to units can misread the prose, and everything downstream inherits whatever it got wrong; the rules constrain what it may **claim**, not whether it read correctly. What the format offers is that from the boundary onward nothing degrades further, and that what did arrive is reviewable by the person it arrives at.

What this buys, both ways

WHAT TO CARRY AWAY

- A claim and its citation travel as one artifact from the model that wrote it to the person who reviews it — nothing is translated, so nothing is lost in translation.
- Confidence falling but never rising across a hop is the Data Processing Inequality, engineered into a document format and checked mechanically, not left to whoever happens to be doing the summarising.
- Fitting a document to a budget is an information bottleneck around the question you actually asked: what the focus claim depends on survives, including its rebuttal, or the operation fails rather than mis-reporting.
- Merging is a mathematical join — commutative, order-independent, and honest about disagreement — computed from what a claim **says**, so two agents recording the same fact need no registry to agree on its identity.
- Across five documents and two models, prose summarising lost 42 of 90 hedges and 49 of 50 citations while compressing **harder** than the format did; on the `smysl` side both are zero by construction, not by care.
- One model call at the entrance, none afterwards: every operation downstream of ingest is deterministic computation over the graph rather than another round of inference.